LTL Runtime Monitoring and Simulation System Technical Overview and Specifications

github@pinhas019/LtlMonitor

Formal Verification & Robotics Team

June 2026

Abstract

This document provides a simple, high-level overview of the **Linear Temporal Logic (LTL) Runtime Monitor and Simulation System**. The system verifies that a robot behaves correctly and safely while executing tasks. It uses formal temporal logic formulas along with a hybrid evaluator that combines quick numerical rules with Large Language Model (LLM) reasoning to assess sensor data.

1 System Architecture

The system is split into two main parts:

1. **LTL State Monitor**: Tracks the formal mathematical states and rules to ensure the task is progressing correctly.
2. **Observation Evaluator**: Translates raw robot sensors (like laser scans and GPS coordinates) into simple true/false statements.

All components run inside isolated Docker containers, communicating automatically using ROS 2.

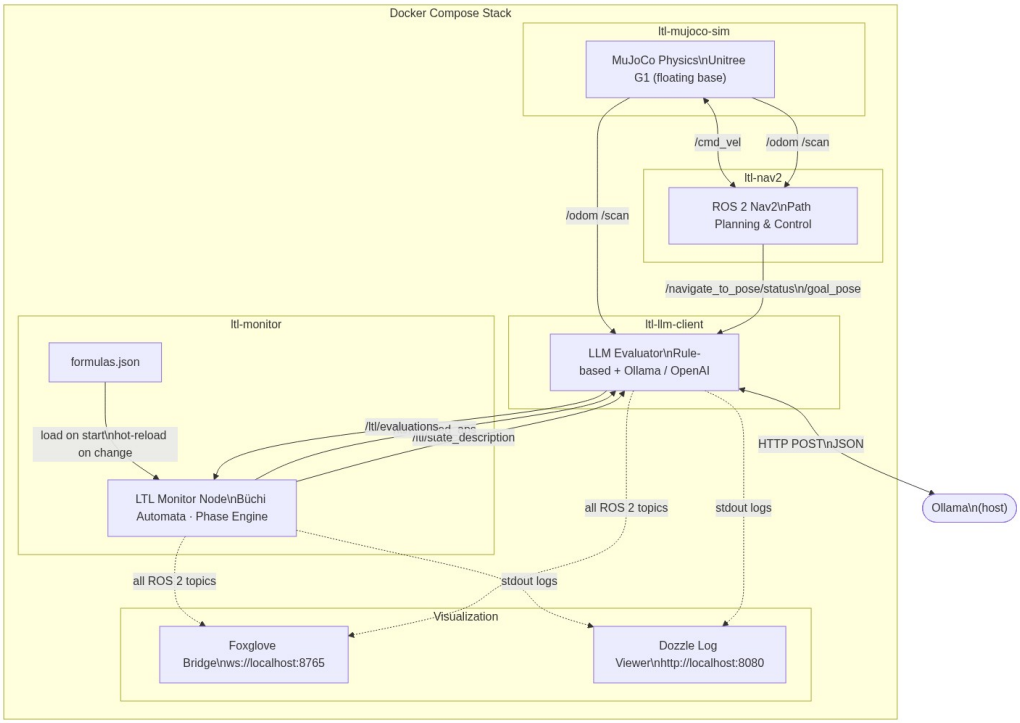

Figure 1: Overview of the System Architecture and Component Connections.

The system consists of six core parts:

- **ltl-monitor**: The central brain. It compiles logic formulas into state transition diagrams (Büchi automata) and tracks execution phases.
- **ltl-llm-client**: The translator. It reads raw sensor topics and determines if conditions like "is moving" or "obstacle nearby" are true.
- **ltl-mujoco-sim**: The simulator. It models the Unitree G1 humanoid robot and generates simulated sensor data (like LiDAR laser scans).
- **ltl-nav2**: The navigator. It handles planning paths and driving the robot around obstacles.
- **ltl-foxglove-bridge**: The visualizer. Streams live data to Foxglove Studio for easy 3D visualization.
- **ltl-dozzle**: The log aggregator. Displays all system logs in a simple web interface.

2 Two-Way Communication Flow

Instead of constantly checking every sensor and logic rule, the system uses a simple, directed two-way communication loop:

1. **Request Active Propositions**: At each step, the **Monitor** looks at its current logic state and figures out exactly which true/false statements (atomic propositions) it needs to evaluate. It publishes this list to the evaluator.
2. **Provide Context**: Along with the required list, the **Monitor** sends a description of the current task phase, terminal goals, and potential failure modes to help the evaluator.

3. **Evaluate Sensors**: The **Evaluator** catches the raw sensor data, evaluates the requested statements using quick math rules first, and uses the LLM as a backup for complex concepts.

4. **Transition States**: The **Evaluator** sends the true/false results back. The **Monitor** updates its logic automata and moves to the next state.

3 Logic Properties and Phase Tracking

The system checks two types of logic rules: **system properties** (goals that must be met) and **named failure modes** (specific errors like crashing).

3.1 System Properties and Named Failure Modes

At runtime, the system monitors two distinct categories of LTL formulas:

- **System Properties (Goals to Meet)**: These are safety or liveness requirements that define task success. For example, a liveness property like $G(\text{active} \rightarrow F(\text{near\_target}))$ ensures that whenever the navigation skill is active, the robot will eventually reach the target.

- **Named Failure Modes (Specific Errors)**: These are parallel safety properties that monitor for specific faults (e.g., crashing or planning failures). For example, a safety property like $G(\neg\text{collision})$ or $G(\neg\text{navigation\_failed})$ is compiled into a separate automaton. If violated, it raises a named fault (such as `SAFETY` or `NAVIGATION`) to trigger an immediate halt, rather than a generic logic violation.

3.2 Formula Monitor Status

Each logic rule tracks a simple state machine to determine if it is satisfied:

- **Inconclusive** (Yellow •): The robot has not finished the task, so we don't know the final outcome yet.

- **Accepted** (Green ✓): The task constraints are currently met.

- **Violated** (Red ×): A safety rule has been broken. The system permanently halts.

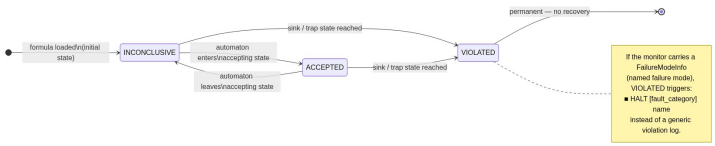

Figure 2: Simple Status Flow of a Monitored LTL Formula.

3.3 Phase Engine and Constraints

To track multi-step tasks (like driving, then picking up an object, then returning), the system splits execution into sequential **phases**. Each phase can enforce four types of checks:

- **Precondition**: Checked once when entering the phase. If not met, the system halts.

- **Invariant**: Checked at every single step. If broken, the system halts immediately.

- **Progress**: Soft constraints. If the robot stops making progress, it registers a violation. If it exceeds a limit, the system goes into `IDLE` to recover.
- **Timing Bounds**: Enforces minimum steps and maximum step limits (timeouts).

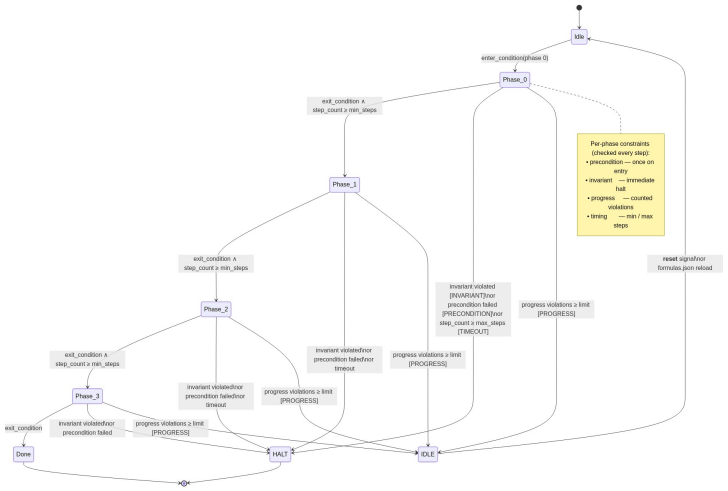

Figure 3: State Machine showing Transitions between Execution Phases.

3.4 Example Generated Automaton

Figure 4 shows an example of a generated Büchi automaton compiled directly by the monitor from LTL properties. This automaton represents a phase-based navigation sequence milestone tracker.

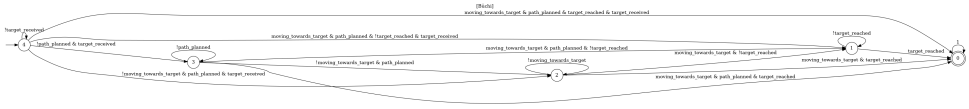

Figure 4: Example of a Compiled Phase Transition Sequence Automaton.

4 Hybrid Evaluation Model

To keep the system fast and responsive, the evaluator node splits its decisions into two paths: a rule-based fast path and an LLM-based slow path.

4.1 Fast Path (Rule-Based Evaluation)

When the client receives the active list of Atomic Propositions (APs), it performs a regular expression search on each AP's description for the pattern `True when <expression>`. If matched, the expression is extracted and evaluated directly against a local dictionary of mapped sensor telemetry (such as `distance_to_target`, `min_range`, and `linear_vel`). This evaluation is run in a sandboxed Python `eval()` environment where all standard built-in functions are removed to prevent arbitrary code execution. It evaluates in a fraction of a millisecond, completely bypassing the LLM.

4.2 Slow Path (LLM Fallback)

If an AP's description cannot be parsed numerically (e.g., subjective concepts like "the robot is oscillating or stuck"), it falls back to the local Large Language Model. The client builds a

structured text prompt including the phase context and sensor values. To avoid blocking the main ROS 2 callback loop, the query is placed into an asynchronous queue and processed by a background worker thread.

5 Core Classes and Code Flow

Behind the scenes, the code is structured around three key concepts:

- **LTLMonitor**: Represents a single logic rule. It loads a compiled automaton and keeps track of its current status.
- **MultiMonitor**: Coordinates multiple monitors running at the same time, merging their lists of required variables.
- **SkillSpec**: Parses the main specification file (`formulas.json`) which defines all the phases, rules, and success/failure conditions.

When sensor updates arrive, the **MultiMonitor** updates all active rule automata. If a critical safety invariant is violated, or a named failure automaton detects an issue, the system alerts the robot to halt immediately. If the success condition is met, the monitoring session completes successfully.

6 Example Walkthrough: GoToTarget

Consider a simple navigation task where the robot drives to a coordinate:

1. **Idle**: The robot awaits a goal.
2. **Initialization**: A goal is received. The system checks preconditions and transitions to active monitoring.
3. **Navigation**: The robot drives. The system verifies safety rules (no obstacles) and checks progress at every step.
4. **GoalReached**: The robot arrives within range, triggers success, and enters the idle state.